



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: _Kenneth Isaías Yahid López Castillo_____ **Carné:** _1138025_____

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Forma de comunicar sistemas por HTTP usando recursos (URLs) y métodos estándar, sin guardar estado entre solicitudes.
2. ¿Qué es un endpoint?
Ruta concreta de la API que ejecuta una operación.
3. ¿Qué diferencia existe entre una URL y un endpoint?
La URL es la dirección completa; el endpoint es la ruta + método que expone un recurso.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
GET = leer,
POST = crear,
PUT = actualizar completo,
PATCH = actualizar parcial,
DELETE = eliminar.
5. ¿Qué es JSON y por qué es común en las APIs?
Es un formato de texto clave-valor, liviano y legible, que entienden todos los lenguajes.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Indican el resultado. 200 = OK, 404 = no encontrado, 500 = error del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
Código de estado, headers y body (los datos o el mensaje de error).
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Probar sin escribir código: enviar solicitudes (GET, POST, PUT, DELETE) directo desde una interfaz, sin armar un programa cliente.
Detectar errores temprano: se ven los códigos HTTP y las respuestas antes de programar, así sabemos si el endpoint o el JSON están mal.
Entender la API: confirmamos qué parámetros pide, qué devuelve y cómo responde ante errores, antes de integrarla.
Guardar las pruebas: las organizamos en colecciones y las reusamos cuando queramos.
Configurar todo fácil: headers, parámetros, body y variables en un solo lugar.
Automatizar y compartir: correr pruebas en serie y exportar la colección para el equipo.



9. ¿Qué es y para qué sirve Swagger UI?

Es una interfaz web que documenta la API y permite probar endpoints con "Try it out".

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Para el cliente sirve para armar, enviar y guardar solicitudes sin programar. Sirve para probar la API antes de codearla, organizar colecciones y automatizar pruebas.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

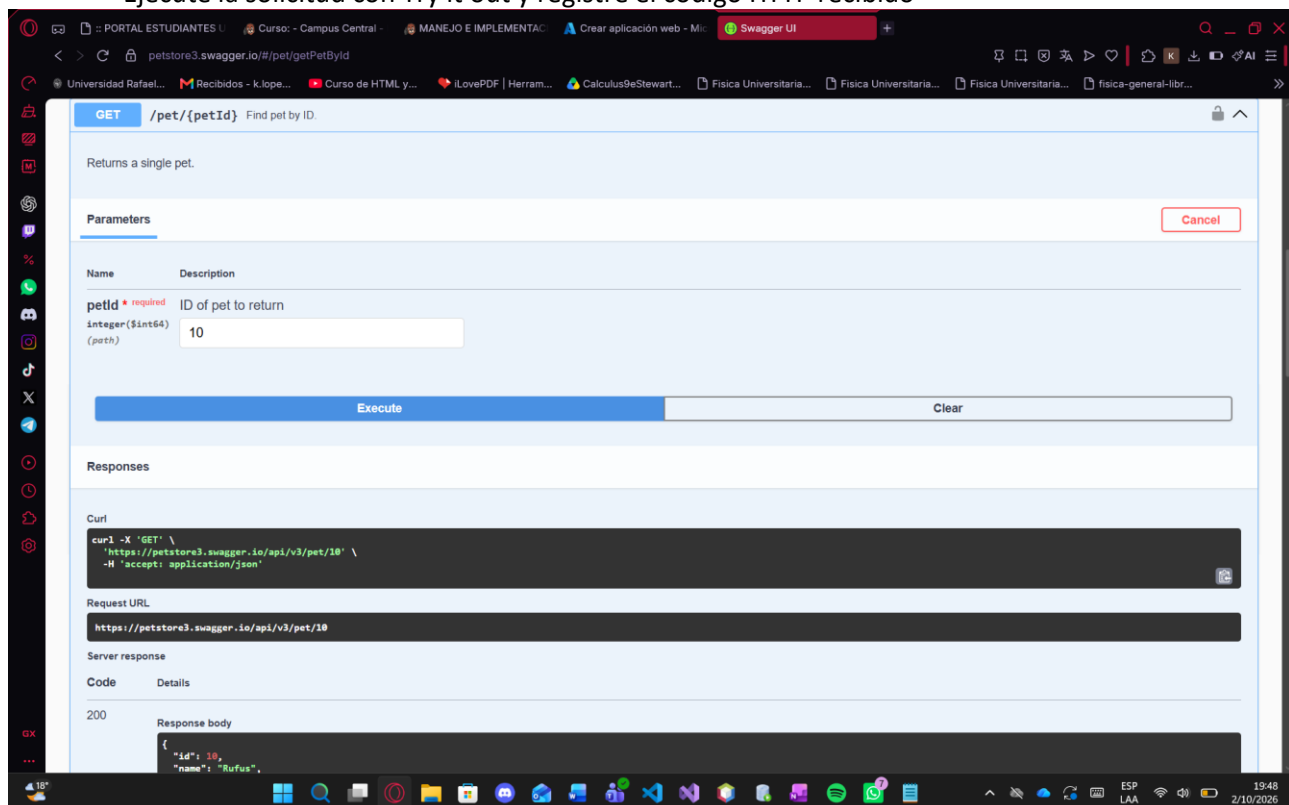
API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

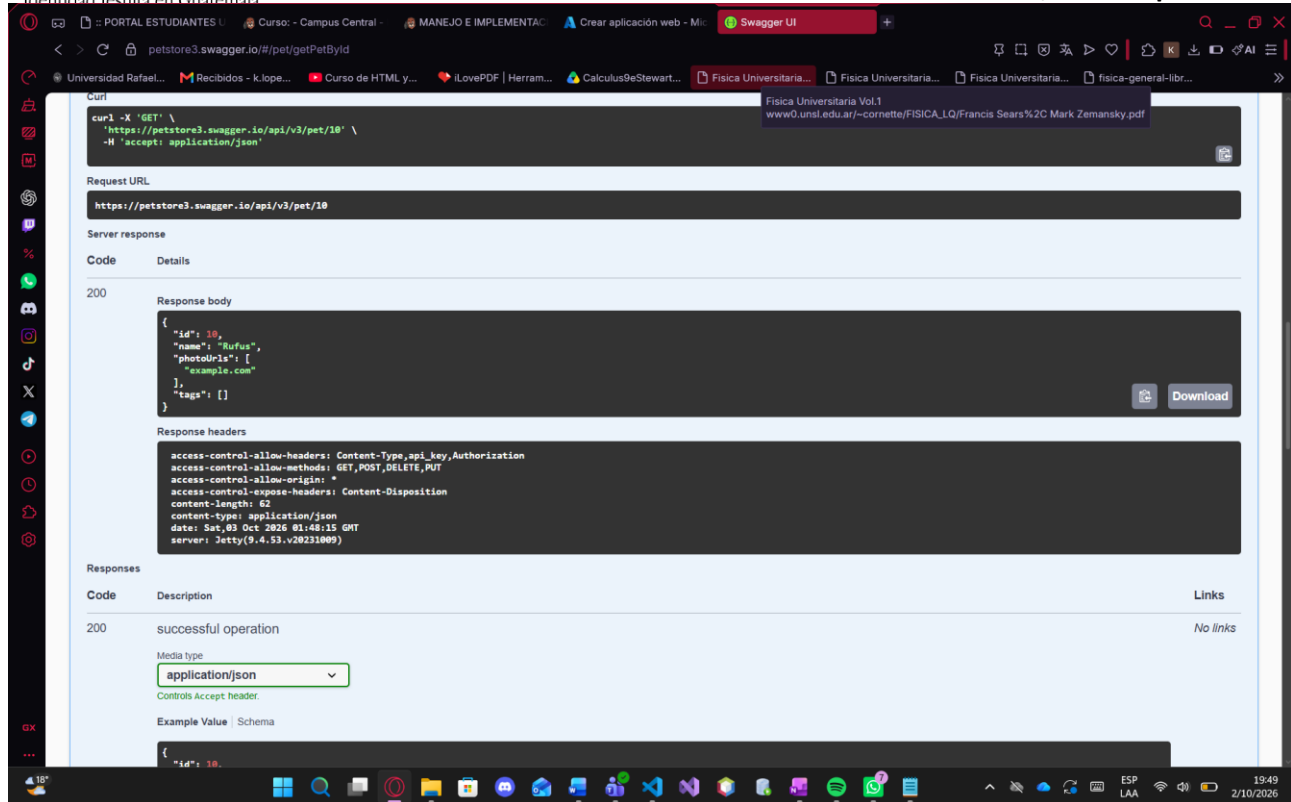
Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID: GET/pet/{petId}
- Indique el método HTTP utilizado: GET
- Indique qué parámetro necesita la solicitud: petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido





- Explique con sus palabras qué información contiene la respuesta: los datos de la mascota: id, name, photoUrls, tags.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



PORTAL ESTUDIANTES | Curso: - Campus Central - | MANEJO E IMPLEMENTAC... | Crear aplicación web - Mic... | Swagger UI

petstore3.swagger.io/#/pet/addPet

PUT /pet Update an existing pet

POST /pet Add a new pet to the store.

Add a new pet to the store.

Parameters

No parameters

Request body **required**

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 1138025,
  "name": "Volter",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota de Kenneth"
    }
  ],
  "status": "available"
}
```

Execute

18°

2/10/2026

PORTAL ESTUDIANTES | Curso: - Campus Central - | MANEJO E IMPLEMENTAC... | Crear aplicación web - Mic... | Swagger UI

petstore3.swagger.io/#/pet/addPet

curl -X 'POST' \

```
'https://petstore3.swagger.io/api/v3/pet' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "id": 1138025,
  "name": "Volter",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota de Kenneth"
    }
  ],
  "status": "available"
}'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 1138025,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Volter",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota de Kenneth"
    }
  ],
  "status": "available"
}
```

Download

18°

2/10/2026



Swagger UI

petstore3.swagger.io/#/pet/addPet

```
curl -X 'POST' \
  https://petstore3.swagger.io/api/v3/pet \
  -H 'accept: application/json' \
  -d '{
    "id": 0,
    "name": "Mascota de Kenneth",
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body <pre>{ "id": 1138025, "category": { "id": 1, "name": "Dogs" }, "name": "Volter", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "Mascota de Kenneth" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 164 content-type: application/json date: Sat, 03 Oct 2026 01:55:20 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description
------	-------------

Swagger UI

petstore3.swagger.io/#/pet/{petId}

GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Name	Description
petId ^{required}	ID of pet to return

integer(int64) (path)

1138025

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  https://petstore3.swagger.io/api/v3/pet/1138025 \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1138025

Server response

Code	Details
200	Response body <pre>{ "id": 1138025, "category": { "id": 1, "name": "Dogs" }, "name": "Volter", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "Mascota de Kenneth" }], "status": "available" }</pre>



Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1138025' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1138025

Server response

Code	Details
200	Response body <pre>{ "id": 1138025, "category": { "id": 1, "name": "Dogs" }, "name": "Volter", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "Mascota de Kenneth" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 154 content-type: application/json date: Sat, 02 Oct 2026 01:56:37 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.

PUT /pet Update an existing pet

Update an existing pet by Id.

Parameters

No parameters

Request body **required**

application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 1138025,
  "name": "Toby",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

Execute

Responses



The screenshot shows the Swagger UI interface for a REST API. The top bar indicates the current endpoint is `petstore3.swagger.io/api/v3/pet`. The request URL is `https://petstore3.swagger.io/api/v3/pet`. The server response is a 200 status code, indicating a successful operation. The response body is a JSON object representing a pet:

```
{
  "id": 1138005,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Toby",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

The response headers are:

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 141
content-type: application/json
date: Sat,03 Oct 2026 02:03:27 GMT
server: Jetty(9.4.53.v20231009)
```

The bottom section shows a table of responses:

Code	Description	Links
200	Successful operation	No links

This screenshot is identical to the one above, showing the Swagger UI interface for a REST API. The top bar indicates the current endpoint is `petstore3.swagger.io/api/v3/pet`. The request URL is `https://petstore3.swagger.io/api/v3/pet`. The server response is a 200 status code, indicating a successful operation. The response body is a JSON object representing a pet:

```
{
  "id": 1138005,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Toby",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

The response headers are:

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 141
content-type: application/json
date: Sat,03 Oct 2026 02:03:27 GMT
server: Jetty(9.4.53.v20231009)
```

The bottom section shows a table of responses:

Code	Description	Links
200	Successful operation	No links

- ¿Qué método HTTP utilizó y por qué?: PUT, porque ese método sirve actualiza un recurso que ya existe y envía el objeto completo.
- ¿Qué datos fueron enviados en el body?: el mismo JSON, cambiando "name": "Toby" y "tags": "Mascota".



- ¿Qué código HTTP devolvió la API?: 200
- ¿Cómo comprobó que el cambio realmente ocurrió?: con GET /pet/1138025 y confirmar que ya salen el nombre y el tag nuevo.

The screenshot shows the Swagger UI interface for the petstore3 API. The endpoint selected is GET /pet/{petId} with the description "Find pet by ID." and "Returns a single pet." The parameter petId is set to 1138025. The "Execute" button is highlighted in blue. Below the "Responses" section, the "Server response" is shown with a status code of 200 and a response body containing {"id": 1138025}.

GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Name	Description
petId required	ID of pet to return

integer(\$int64) (path)

1138025

Execute Clear

Responses

curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/1138025' \ -H 'accept: application/json'

Request URL

https://petstore3.swagger.io/api/v3/pet/1138025

Server response

Code	Details
200	Response body

Response body

```
{ "id": 1138025 }
```

This screenshot is identical to the one above, showing the Swagger UI interface for the petstore3 API. The endpoint selected is GET /pet/{petId} with the description "Find pet by ID." and "Returns a single pet." The parameter petId is set to 1138025. The "Execute" button is highlighted in blue. Below the "Responses" section, the "Server response" is shown with a status code of 200 and a response body containing {"id": 1138025}.

GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Name	Description
petId required	ID of pet to return

integer(\$int64) (path)

1138025

Execute Clear

Responses

curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/1138025' \ -H 'accept: application/json'

Request URL

https://petstore3.swagger.io/api/v3/pet/1138025

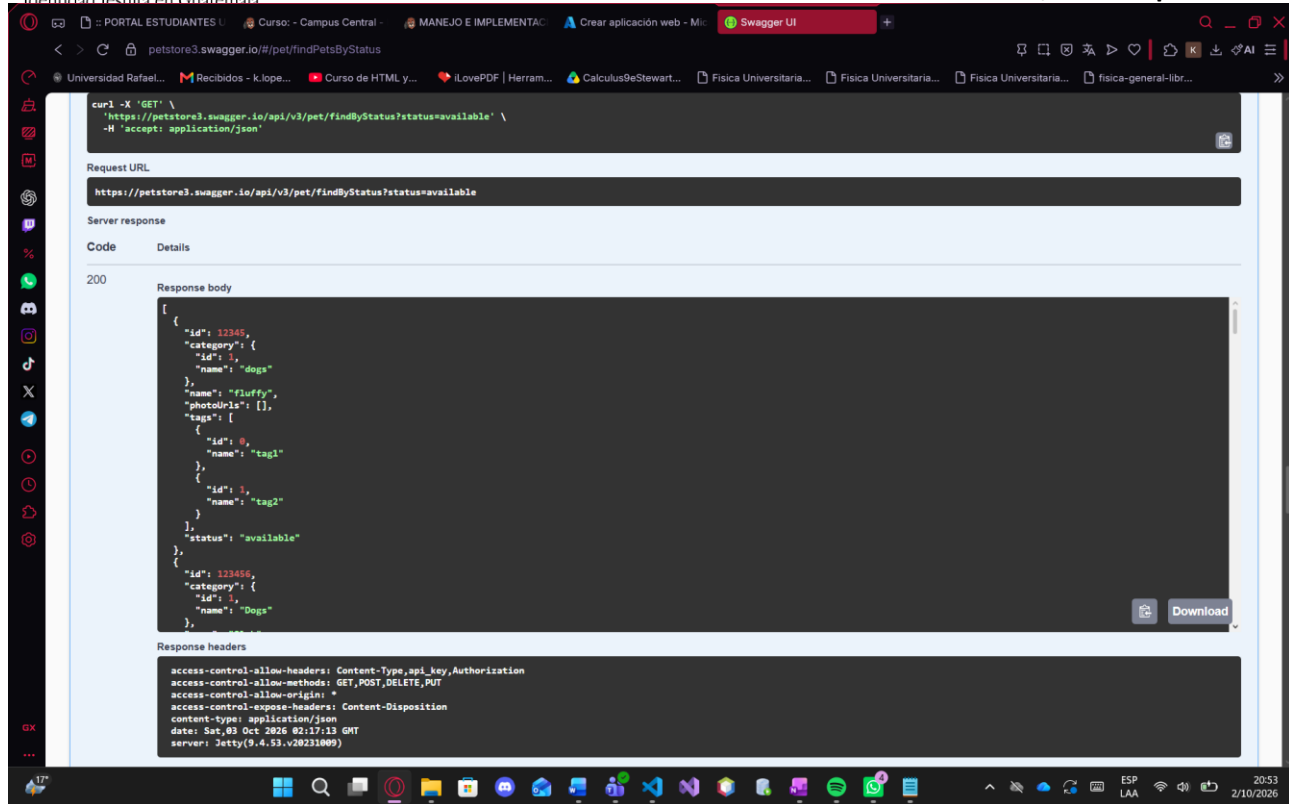
Server response

Code	Details
200	Response body

Response body

```
{ "id": 1138025 }
```

The screenshot displays the Swagger UI for the `/pet/findPetsByStatus` endpoint. The interface includes a header with the endpoint name and a description: "Finds Pets by status. Multiple status values can be provided with comma separated strings." Below this, the "Parameters" section shows a single parameter: `status`, which is required and of type `string` (query). The parameter value is set to `available`. A blue "Execute" button is visible. The "Responses" section shows the curl command, the request URL, and the server response, which is a 200 status code with a response body.



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/10`. The request headers include `Accept: application/json`. The response is a 200 OK status with a JSON body:

```
{
  "id": 10,
  "name": "RuIus",
  "photoUrls": [
    "example.com"
  ],
  "tags": []
}
```

The left sidebar shows the 'COLLECTIONS' list with 'MIA_SEMANA13' expanded, containing 'POST Login', 'GET Usuario Autenticado', 'GET https://postman-echo.com/get', and 'MIA_SEMANA13_API' (expanded) with 'GET Consulta por ID' selected.

POST – Crear mascota

The screenshot shows the Postman interface with a POST request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object:

```
{
  "id": 1138825,
  "name": "Volter",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Mascota de Kenneth"
    }
  ]
}
```

The left sidebar shows the 'COLLECTIONS' list with 'MIA_SEMANA13' expanded, containing 'POST Login', 'GET Usuario Autenticado', 'GET https://postman-echo.com/get', and 'MIA_SEMANA13_API' (expanded) with 'GET Consulta por ID' and 'POST Crear Mascota' (selected).



The screenshot shows the Postman interface with a POST request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object representing a pet named "Volter" in the "Dogs" category. The response is a 200 OK status with a 277 ms response time and a 516 B body. The response body is a JSON object that includes the pet's details and a tag.

```
1 {
2   "id": 1138025,
3   "name": "Volter",
4   "category": {
5     "id": 1,
6     "name": "Dogs"
7   },
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 0,
14      "name": "Mascota de Kenneth"
15    }
16  ],
17  "status": "available"
18 }
```

PUT – Modificar mascota

The screenshot shows the Postman interface with a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object representing a pet named "Toby" in the "Dogs" category. The response is a 200 OK status with a 65 ms response time and a 503 B body. The response body is a JSON object that includes the pet's details and a tag.

```
1 {
2   "id": 1138025,
3   "category": {
4     "id": 1,
5     "name": "Dogs"
6   },
7   "name": "Toby",
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 0,
14      "name": "Mascota"
15    }
16  ],
17  "status": "available"
18 }
```



The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available`. The response is a 200 OK status with a JSON body containing pet details.

Query Params

Key	Value	Description
status	available	

Body

```
{
  "id": 4,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Dog 1",
  "photoUrls": [
    "url1",
    "url2"
  ],
  "tags": [
    {
      "id": 1,
      "name": "tag1"
    }
  ]
}
```

DELETE – Eliminar mascota

The screenshot shows the Postman interface with a DELETE request to `https://petstore3.swagger.io/api/v3/pet/20260930`. The response is a 200 OK status with a JSON body indicating the pet was deleted.

Body

```
{
  "message": "Pet deleted"
}
```



Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

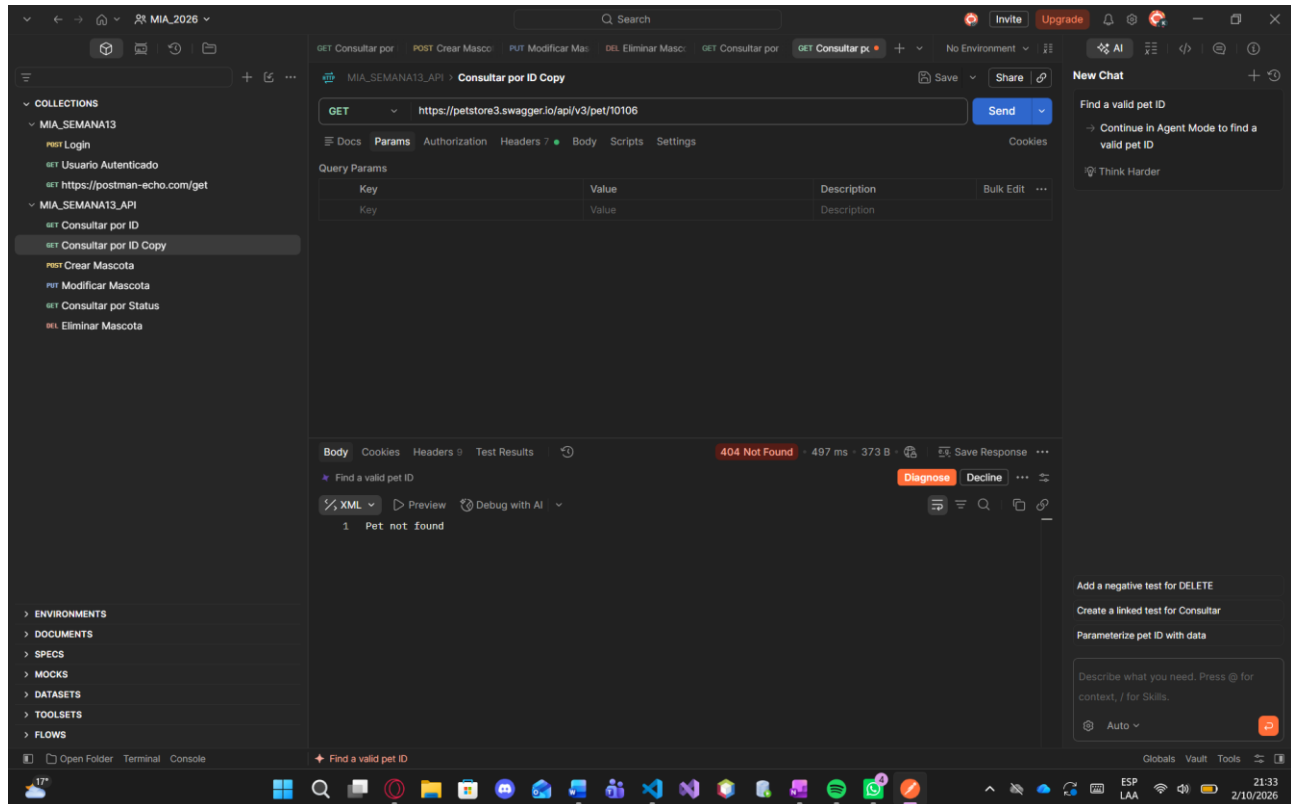
Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

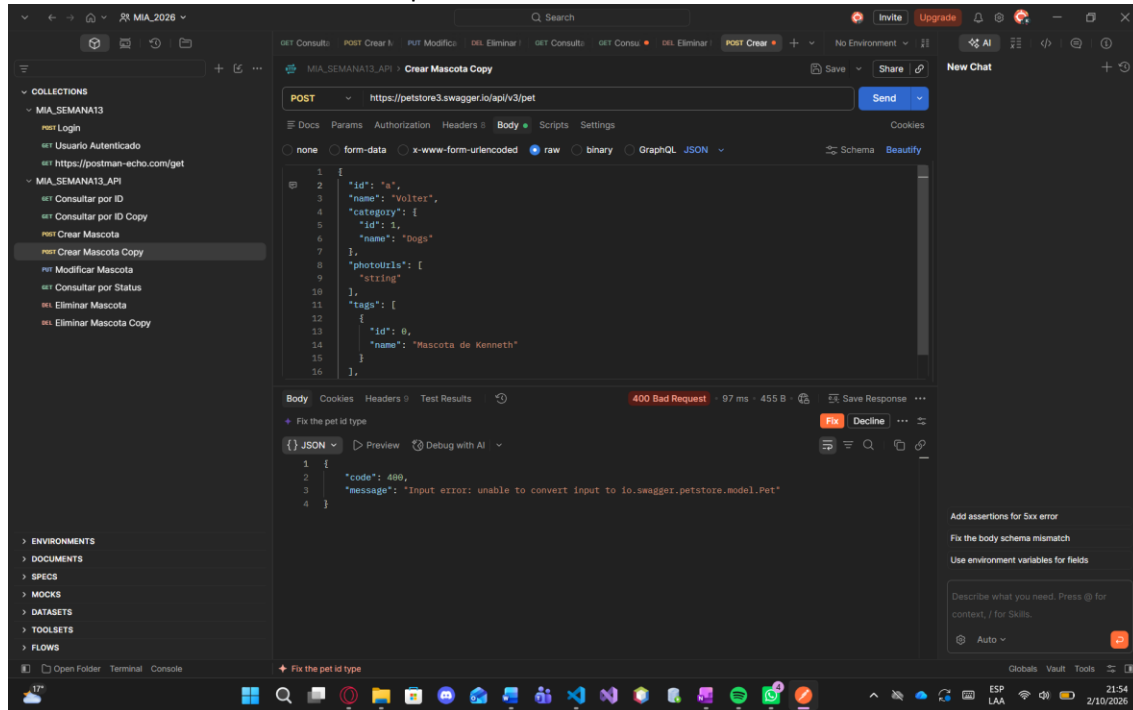
Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

1. Consultar un ID inexistente

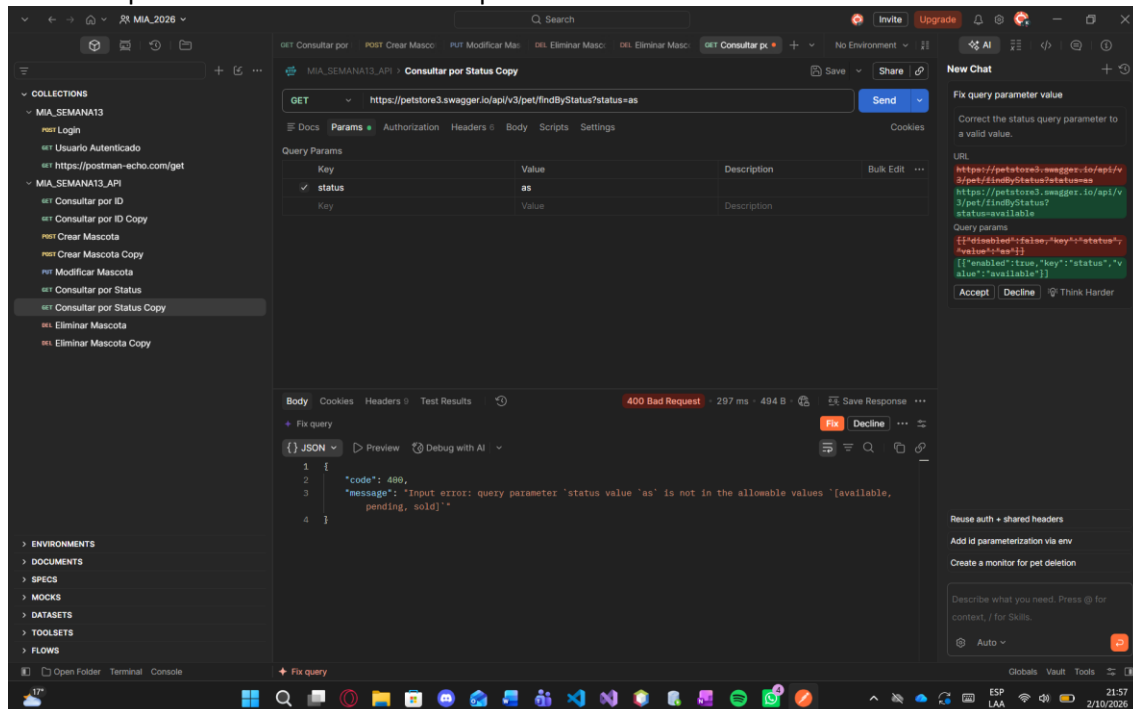


- a) Solicitud realizada: GET <https://petstore3.swagger.io/api/v3/pet/10106>
- b) Código HTTP recibido: 404
- c) Respuesta de la API: Pet not found
- d) Qué esperaba que ocurriera: que la API avise que no existe esa mascota.
- e) Qué ocurrió realmente: devolvió el error 404.
- f) Explicación de la causa: El ID no existe en el servidor, entonces no hay recurso que devolver.



- a) Solicitud realizada: POST <https://petstore3.swagger.io/api/v3/pet>
- b) Código HTTP recibido: 400
- c) Respuesta de la API: Input error: unable to convert input to io.swagger.petstore.model.Pet
- d) Qué esperaba que ocurriera: que rechace la solicitud por datos inválidos.
- e) Qué ocurrió realmente: devolvió el error y no creó la mascota.
- f) Explicación de la causa: El campo id debe ser numérico y el servidor no pudo convertir el texto al objeto Pet.

3. Utilizar un parámetro con un valor no esperado.





- Solicitud realizada: GET <https://petstore3.swagger.io/api/v3/pet/findByStatus?status=as>
- Código HTTP recibido: 400
- Respuesta de la API: Input error: query parameter `status` value `as` is not in the allowable values `[available, pending, sold]`
- Qué esperaba que ocurriera: que rechace un status que no existe.
- Qué ocurrió realmente: devolvió el error.
- Explicación de la causa: solo aceptan available, pending y sold.

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

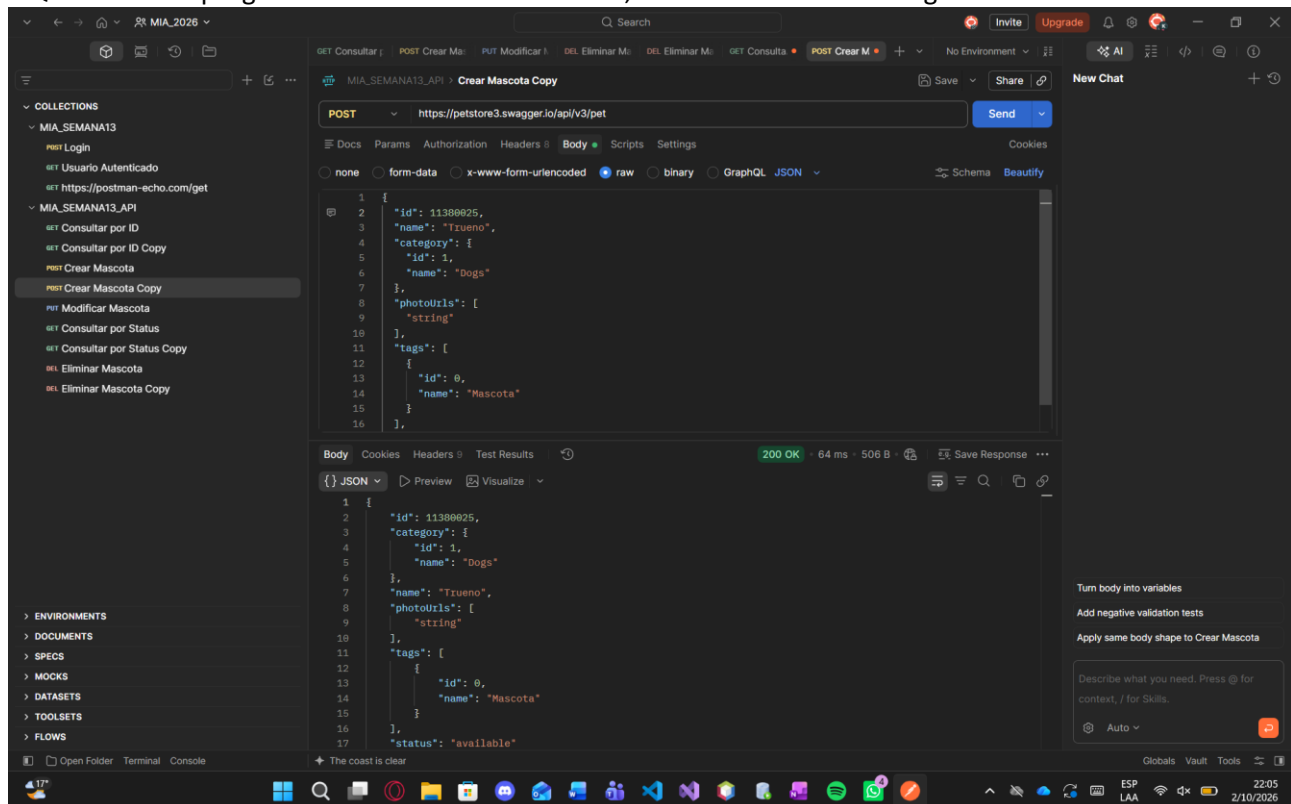
Capture evidencia de cada paso.

Registrar:

Endpoint: /pet

Método: POST

Qué analiza el programa: Crea una nueva mascota, al terminar lanza el código 202 de éxito.

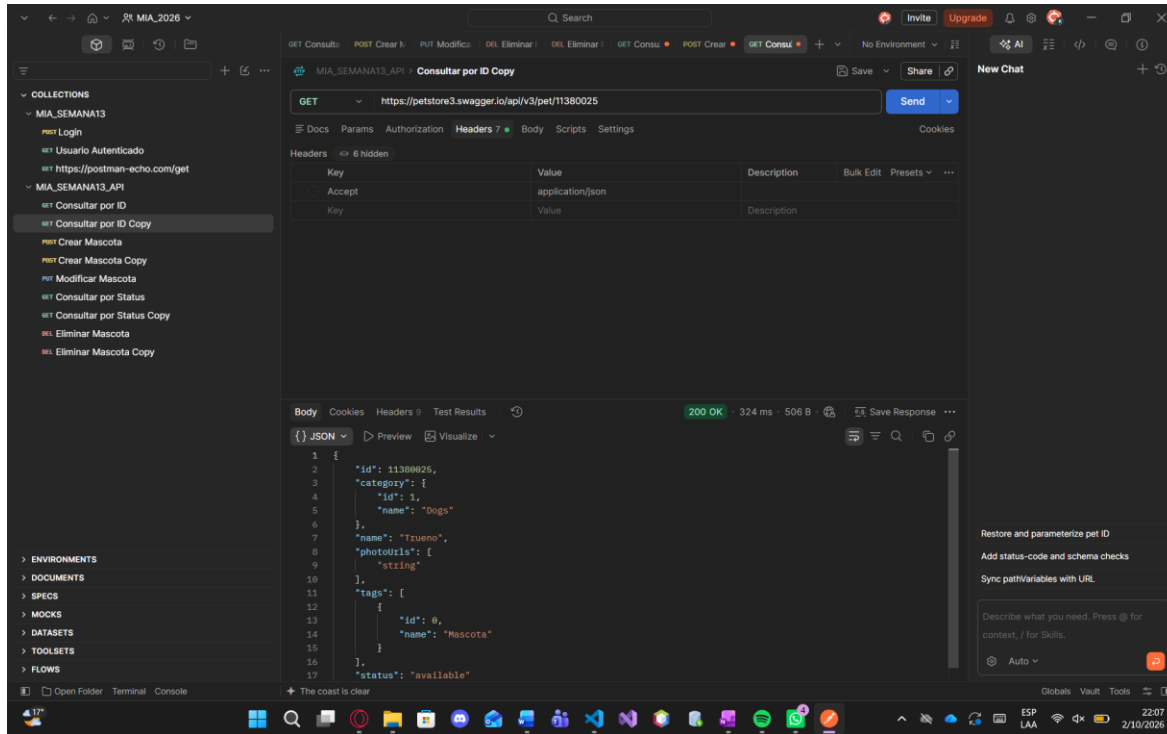




Endpoint: /pet/{id}

Método: GET

Qué analiza el programa: Busca la nueva mascota creada y devuelve sus datos, al terminar lanza el código 202 de éxito.

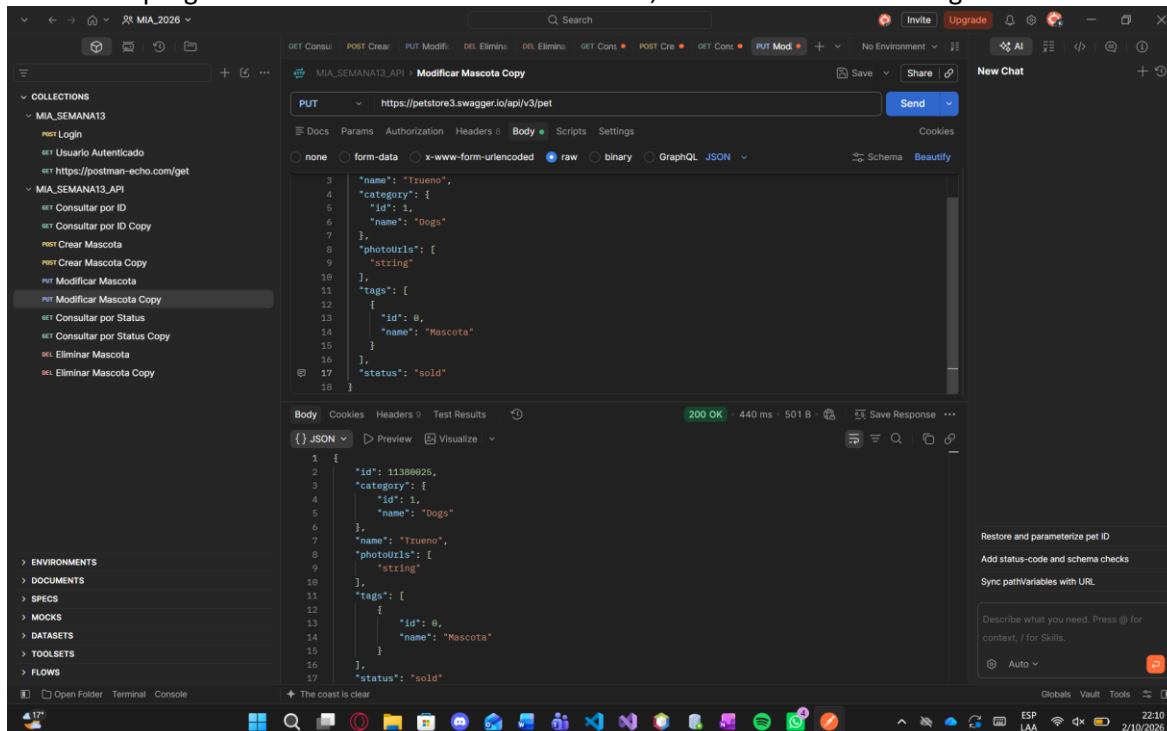


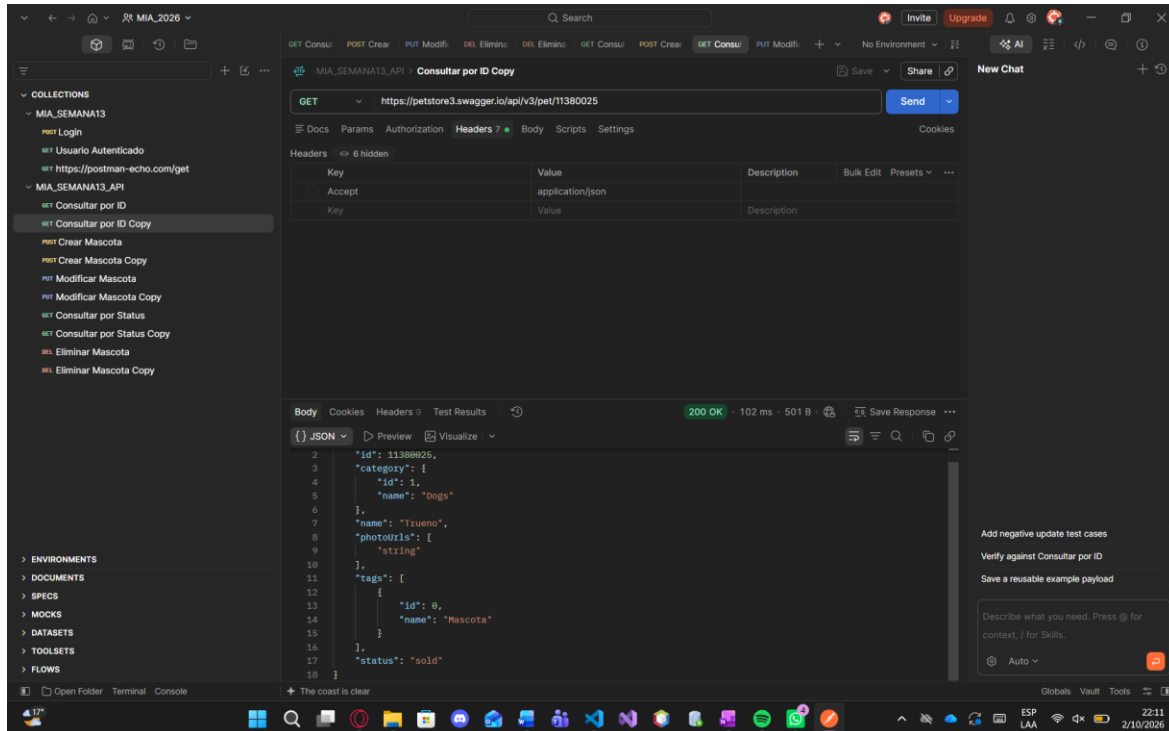
Actualizar estado:

Endpoint: /pet

Método: PUT

Qué analiza el programa: Cambia el estado de la mascota, al terminar lanza el código 202 de éxito.



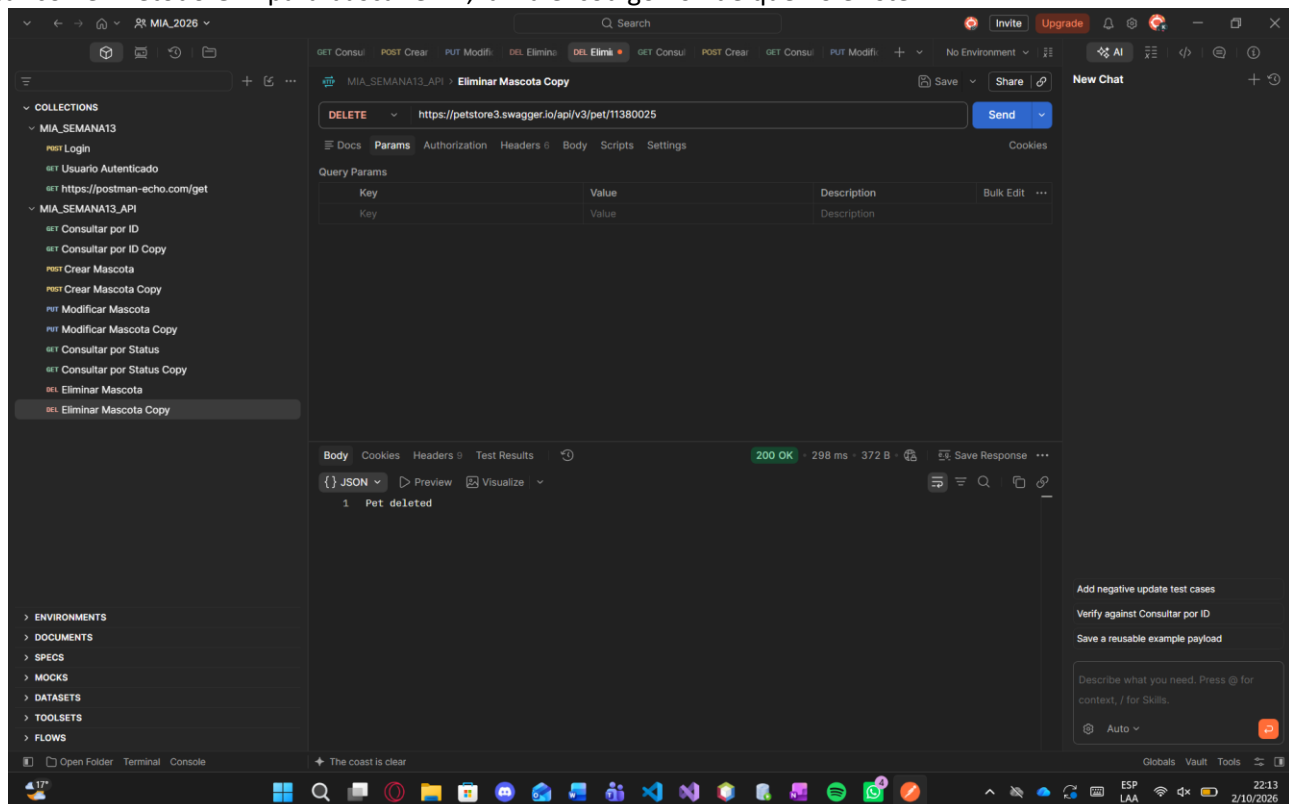


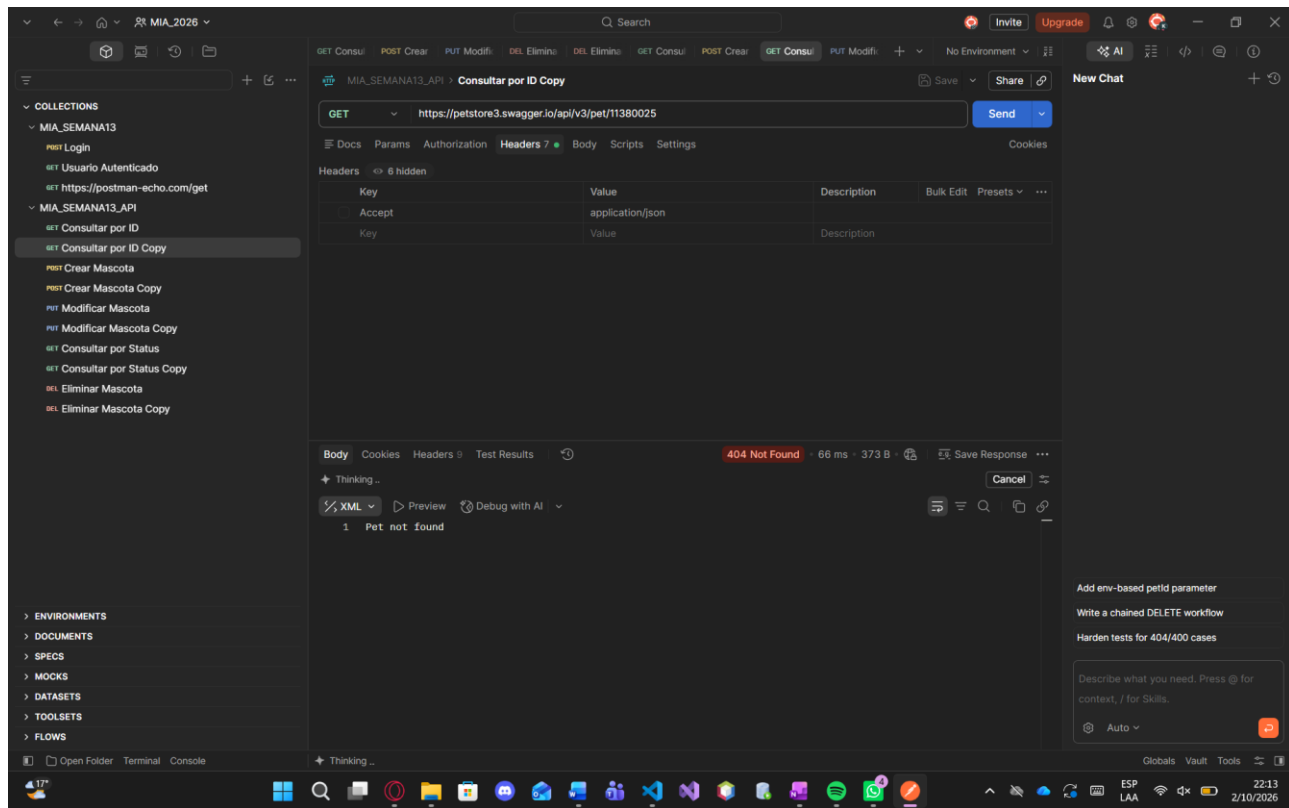
Eliminar

Endpoint: `/pet/{id}`

Método: DELETE

Qué analiza el programa: Elimina el registro de la mascota con ese ID, al terminar lanza el código 202. Y al verificar con el método GET para buscar el ID, lanza el código 404 de que no existe.





Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.